

Teach Module 50: Capstone Build - Frontend and Persistence

This note is based on Module 50's topic title from the bootcamp outline, but the teaching content is original and does not use the course material as the lesson source.

Module 50 Overview

Module 50 is about connecting the visible part of the capstone application to real backend APIs and a real database. At this stage, the project should stop behaving like a mockup and start behaving like a working full-stack system.

The main topics are:

- React frontend connected to Spring Boot APIs
- Oracle persistence with Spring Data JPA
- End-to-end data flow validation
- UI testing with Selenium
- Internal review and refinement
- Capstone frontend and persistence lab

Big Picture

In this module, your capstone becomes a real full-stack application.

```
React UI -> Spring Boot REST API -> Spring Data JPA -> Oracle Database
```

Think of the flow like this:

```
User fills form in React
React sends a request to Spring Boot
Spring Boot validates and processes the request
Spring Data JPA saves the data
Oracle stores the row
React reloads or updates the displayed data
```

The goal is not only to create screens or endpoints. The goal is to prove that data can travel through the entire system correctly.

React Frontend Connected To Spring Boot APIs

React should focus on user interaction:

- Display data
- Collect input
- Call backend APIs
- Show loading, success, and error states
- Update the screen when data changes

React should not contain the main business rules. Business rules belong in the backend service layer.

A simple React API function might look like this:

```

async function createCustomer(customer) {
  const response = await fetch("http://localhost:8080/api/customers", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(customer)
  });

  if (!response.ok) {
    throw new Error("Failed to create customer");
  }

  return response.json();
}

```

A React form can call that function when the user submits:

```

const handleSubmit = async (event) => {
  event.preventDefault();

  const savedCustomer = await createCustomer({
    name,
    email
  });

  setCustomers([...customers, savedCustomer]);
};

```

A clean frontend structure might look like this:

```

src/
  api/
    customersApi.js
  components/
    CustomerForm.jsx
    CustomerList.jsx
  pages/
    CustomersPage.jsx

```

The API file handles HTTP calls. Components handle display and interaction. Pages assemble the feature.

Oracle Persistence With Spring Data JPA

Spring Data JPA maps Java objects to database tables. In a capstone project, this is how your backend persists data instead of losing it when the app restarts.

Example entity:

```

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "customers")
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String email;

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}

```

Repository:

```

import org.springframework.data.jpa.repository.JpaRepository;

public interface CustomerRepository extends JpaRepository<Customer, Long> {
}

```

Service:

```
import java.util.List;
import org.springframework.stereotype.Service;

@Service
public class CustomerService {

    private final CustomerRepository customerRepository;

    public CustomerService(CustomerRepository customerRepository) {
        this.customerRepository = customerRepository;
    }

    public Customer create(Customer customer) {
        return customerRepository.save(customer);
    }

    public List<Customer> findAll() {
        return customerRepository.findAll();
    }
}
```

Controller:

```
import java.util.List;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/customers")
@CrossOrigin(origins = "http://localhost:3000")
public class CustomerController {

    private final CustomerService customerService;

    public CustomerController(CustomerService customerService) {
        this.customerService = customerService;
    }

    @PostMapping
    public Customer create(@RequestBody Customer customer) {
        return customerService.create(customer);
    }

    @GetMapping
    public List<Customer> findAll() {
```

```
        return customerService.findAll();
    }
}
```

The important rule is simple:

```
React never talks directly to Oracle.
React talks to Spring Boot.
Spring Boot talks to Oracle.
```

End-To-End Data Flow Validation

End-to-end validation means testing the full journey from browser to database and back.

Example:

```
Create customer in React
Confirm POST /api/customers succeeds
Confirm Oracle row exists
Confirm GET /api/customers returns the new customer
Confirm React displays the new customer
```

You should validate at multiple layers:

- Frontend: Does the button work?
- API: Does the endpoint return correct JSON?
- Service: Is the business logic correct?
- Database: Was the record actually saved?
- UI refresh: Does the new data appear?

Useful debugging tools:

- Browser DevTools console
- Browser DevTools Network tab
- Spring Boot logs
- Postman or curl
- SQL query against Oracle
- Repository or integration tests

Common bugs in this module:

- CORS error between React and Spring Boot
- Wrong API URL
- JSON field mismatch, such as `customerName` in React but `name` in Java
- JPA table or column mismatch
- Oracle connection issue
- Backend saves data but frontend state does not update
- Frontend still displays mock data instead of API data

UI Testing With Selenium

Selenium tests the application like a real user.

Example test scenario:

```
Open customer page
Type name and email
Click Save
Verify customer appears in list
```

Simple Selenium test:

```
import static org.junit.jupiter.api.Assertions.assertTrue;

import org.junit.jupiter.api.Test;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class CustomerUiTest {

    @Test
    void userCanCreateCustomer() {
        WebDriver driver = new ChromeDriver();

        driver.get("http://localhost:3000/customers");

        driver.findElement(By.id("name")).sendKeys("Ava Patel");
        driver.findElement(By.id("email")).sendKeys("ava@example.com");
        driver.findElement(By.id("saveCustomer")).click();

        WebElement list = driver.findElement(By.id("customerList"));

        assertTrue(list.getText().contains("Ava Patel"));

        driver.quit();
    }
}
```

Use stable selectors in the React page:

```
<input id="name" />
<input id="email" />
<button id="saveCustomer">Save</button>
<div id="customerList"></div>
```

Avoid relying on fragile selectors such as long CSS paths.

Internal Review And Refinement

Before calling a capstone feature complete, review it like an engineer.

Ask:

- Does the UI handle errors?
- Does the UI show loading states?
- Does the backend validate input?
- Are API names consistent?
- Is database persistence working?
- Are secrets excluded from source code?
- Are tests meaningful?
- Can another person run the app from the README?

A polished capstone feature should have:

- Working React screen
- Real API integration
- Spring Boot controller, service, and repository structure
- Oracle persistence
- Basic validation
- Useful error handling
- At least one UI automation test
- Clear demo path

Practice Exercises

Exercise 1: Build One Full CRUD Feature

Create a full-stack feature for one domain object.

Example choices:

```
Customer
Product
Ticket
BankAccount
Order
Employee
```

Required:

```
React list page
React create form
Spring Boot REST controller
Service layer
Spring Data JPA repository
Oracle-backed entity/table
GET all records
POST new record
PUT update record
DELETE record
```

Goal: prove you can move data from UI to database and back.

Exercise 2: React API Integration Practice

Create a React page that calls these backend endpoints:

```
GET /api/products
POST /api/products
GET /api/products/{id}
DELETE /api/products/{id}
```

Practice:

- Loading state
- Error state
- Empty list state
- Successful save message
- Form reset after submit

Stretch goal: move all `fetch()` calls into a separate file:

```
src/api/productsApi.js
```

Exercise 3: Oracle Persistence With JPA

Create a JPA entity called `Ticket`.

Fields:

```
id
title
description
status
priority
createdAt
```

Practice:

- Map it with `@Entity`
- Create `TicketRepository`
- Create `TicketService`
- Create `TicketController`
- Persist records into Oracle
- Query all open tickets
- Query tickets by priority

Example repository method:

```
List<Ticket> findByStatus(String status);
```

Exercise 4: End-To-End Data Flow Trace

Pick one feature and document the full request path.

Example:

```
User clicks Save Ticket
React sends POST /api/tickets
Spring controller receives JSON
Service validates ticket
Repository saves entity
Oracle inserts row
Backend returns saved ticket
React updates ticket list
```

Then test each step manually.

Use:

- Browser DevTools Network tab
- Spring Boot logs
- Postman or curl
- SQL query against Oracle
- React UI confirmation

Goal: become good at debugging full-stack problems.

Exercise 5: Validation And Error Handling

Add validation to a create form.

Rules:

```
Name is required
Email must look valid
Status must be selected
Description must be at least 10 characters
```

Backend should reject invalid input.

Example Java validation:

```
@NotBlank
private String name;

@email
private String email;
```

Frontend should show useful error messages.

Practice:

- Submit empty form
- Submit invalid email
- Submit valid data
- Confirm invalid records are not saved

Exercise 6: Selenium UI Test

Write a Selenium test for one complete user flow.

Scenario:

```
Open React page
Click Add Ticket
Fill title
Fill description
Select priority
Click Save
Verify ticket appears in list
```

Good selectors:

```
<input id="ticket-title" />
<textarea id="ticket-description"></textarea>
<button id="save-ticket">Save</button>
<div id="ticket-list"></div>
```

Goal: test the app from the user's point of view.

Exercise 7: Search And Filter UI

Add search or filter behavior to your React list.

Options:

- Search customers by name
- Filter tickets by status
- Filter products by category
- Sort orders by date

Backend version:

```
GET /api/tickets?status=OPEN
```

Frontend version:

```
Use dropdown or search box
Call API when filter changes
Show filtered results
```

Stretch goal: debounce search input.

Exercise 8: Capstone Review Checklist

Review your feature using this checklist:

```
Can the app create a record?
Can the app read records from Oracle?
```

```
Can the app update a record?
Can the app delete a record?
Does the UI show loading states?
Does the UI show errors?
Does backend validation work?
Are API names consistent?
Are DTOs or entities clean?
Is there at least one automated UI test?
Can another developer run it from README instructions?
```

Recommended Practice Sequence

1. JPA entity and repository
2. Spring Boot controller
3. Manual API test
4. React list page
5. React create form
6. Error handling
7. Selenium test
8. Internal review

Module 50 Lab: Ticket Tracker

Lab Goal

Build one complete capstone feature where React, Spring Boot, Spring Data JPA, Oracle, and Selenium all work together.

Use this feature:

```
Ticket Tracker
```

By the end, you should be able to:

```
Create a ticket from React
Save it through Spring Boot
Persist it in Oracle
Display saved tickets in React
Test the flow with Selenium
```

Lab Scenario

Your capstone team needs a simple support ticket system. Users should be able to create tickets and view all submitted tickets.

A ticket has:

```
id
title
```

```
description
priority
status
createdAt
```

Example values:

```
title: Login issue
description: User cannot log in after password reset
priority: HIGH
status: OPEN
createdAt: current timestamp
```

Part 1: Backend Entity

Create a JPA entity named `Ticket` .

```
import java.time.LocalDateTime;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.PrePersist;
import jakarta.persistence.Table;

@Entity
@Table(name = "tickets")
public class Ticket {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;

    private String description;

    private String priority;

    private String status;

    private LocalDateTime createdAt;

    @PrePersist
    public void onCreate() {
        this.createdAt = LocalDateTime.now();
        this.status = "OPEN";
    }
}
```

```
// getters and setters  
}
```

Part 2: Repository

Create `TicketRepository` .

```
import java.util.List;  
import org.springframework.data.jpa.repository.JpaRepository;  
  
public interface TicketRepository extends JpaRepository<Ticket, Long> {  
    List<Ticket> findByStatus(String status);  
    List<Ticket> findByPriority(String priority);  
}
```

Part 3: Service Layer

Create `TicketService` .

```
import java.util.List;  
import org.springframework.stereotype.Service;  
  
@Service  
public class TicketService {  
  
    private final TicketRepository ticketRepository;  
  
    public TicketService(TicketRepository ticketRepository) {  
        this.ticketRepository = ticketRepository;  
    }  
  
    public Ticket createTicket(Ticket ticket) {  
        return ticketRepository.save(ticket);  
    }  
  
    public List<Ticket> getAllTickets() {  
        return ticketRepository.findAll();  
    }  
}
```

Part 4: REST Controller

Create `TicketController` .

```
import java.util.List;  
import org.springframework.web.bind.annotation.CrossOrigin;  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.PostMapping;  
import org.springframework.web.bind.annotation.RequestBody;  
import org.springframework.web.bind.annotation.RequestMapping;
```

```
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/tickets")
@CrossOrigin(origins = "http://localhost:3000")
public class TicketController {

    private final TicketService ticketService;

    public TicketController(TicketService ticketService) {
        this.ticketService = ticketService;
    }

    @PostMapping
    public Ticket createTicket(@RequestBody Ticket ticket) {
        return ticketService.createTicket(ticket);
    }

    @GetMapping
    public List<Ticket> getAllTickets() {
        return ticketService.getAllTickets();
    }
}
```

Test backend manually:

```
POST http://localhost:8080/api/tickets
GET http://localhost:8080/api/tickets
```

Sample POST body:

```
{
  "title": "Login issue",
  "description": "User cannot log in after password reset",
  "priority": "HIGH"
}
```

Part 5: React API File

Create:

```
src/api/ticketsApi.js
```

```
const API_URL = "http://localhost:8080/api/tickets";

export async function getTickets() {
  const response = await fetch(API_URL);

  if (!response.ok) {
```

```

        throw new Error("Failed to load tickets");
    }

    return response.json();
}

export async function createTicket(ticket) {
    const response = await fetch(API_URL, {
        method: "POST",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify(ticket)
    });

    if (!response.ok) {
        throw new Error("Failed to create ticket");
    }

    return response.json();
}

```

Part 6: React Ticket Page

Build a page with:

- Title input
- Description textarea
- Priority dropdown
- Submit button
- Ticket list
- Loading message
- Error message

Required UI behavior:

```

When page loads, fetch all tickets
When user submits form, create ticket
After save, refresh ticket list
Show error if API call fails
Clear form after successful save

```

Part 7: Oracle Verification

After creating a ticket from React, verify the database.

Run a query similar to:

```

SELECT id, title, description, priority, status, created_at
FROM tickets;

```

Confirm:

- The row exists
- The title matches the React form
- The priority matches
- The status is `OPEN`
- The `createdAt` value exists

Part 8: Selenium UI Test

Create a Selenium test for the full user flow.

Test scenario:

```
Open ticket page
Enter title
Enter description
Select priority
Click submit
Verify new ticket appears in list
```

Example test outline:

```
import static org.junit.jupiter.api.Assertions.assertTrue;

import org.junit.jupiter.api.Test;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class TicketUiTest {

    @Test
    void userCanCreateTicketFromUi() {
        WebDriver driver = new ChromeDriver();

        driver.get("http://localhost:3000/tickets");

        driver.findElement(By.id("ticket-title")).sendKeys("Login issue");
        driver.findElement(By.id("ticket-description"))
            .sendKeys("User cannot log in after password reset");
        driver.findElement(By.id("ticket-priority")).sendKeys("HIGH");
        driver.findElement(By.id("save-ticket")).click();

        WebElement ticketList = driver.findElement(By.id("ticket-list"));

        assertTrue(ticketList.getText().contains("Login issue"));

        driver.quit();
    }
}
```


Completion Checklist

Your lab is complete when:

```
React form saves a ticket
Spring Boot receives the request
Ticket is saved in Oracle
React displays saved tickets
Backend GET endpoint returns tickets
Selenium test passes
Errors are handled in the UI
```

Stretch Tasks

Add one or more:

- Update ticket status from `OPEN` to `CLOSED`
- Filter tickets by priority
- Filter tickets by status
- Delete a ticket
- Add backend validation
- Add frontend form validation
- Show success notification after save

Key Takeaway

A capstone feature is only real when the browser, API, service logic, database, and test all agree with each other.